PROGRAMAÇÃO WEB

Revisão para múltipla escolha • Azul: conceitos-chave | Preto: explicações e exemplos

1. Fundamentos, carreira e ferramentas | S1-4

HTML estrutura; CSS apresenta; JavaScript programa comportamento. Front-end atua na interface e nas interações no cliente; back-end processa regras e dados no servidor; full-stack atua nos dois. JS também pode executar no servidor. Perfil T: conhecimentos amplos + especialização.

Ágil: ciclos, entregas e feedback. **Cascata:** etapas predominantemente sequenciais. **DevOps:** integração entre desenvolvimento e operações. Testes automatizados verificam comportamento; revisão de código melhora o trabalho. **Git:** controle de versões, histórico e colaboração; não é hospedagem. MVC separa modelo (dados/regras), visão (apresentação) e controlador (coordenação).

Biblioteca: seu código chama funcionalidades prontas. **Framework:** fornece uma estrutura que coordena partes da aplicação. **API:** interface/contrato para comunicação entre softwares. Acessibilidade: HTML semântico, rótulos, texto alternativo e uso por teclado.

DevTools: Elements inspeciona DOM e estilos; Console exibe mensagens/erros e executa JS; Network mostra requisições, respostas, status e tempos. Editor escreve código; navegador interpreta/renderiza a página.

2. Redes, HTTP, URL e publicação | S2-4

Internet é a infraestrutura de redes; **Web** é um serviço que a utiliza. IP endereça/encaminha pacotes; TCP fornece transporte confiável e ordenado. **DNS** resolve nomes em registros, incluindo endereços IP. Domínio é o nome; hospedagem disponibiliza arquivos/aplicação; Apache é servidor web.

HTTP: cliente envia requisição e servidor devolve resposta. GET solicita recurso; POST envia dados para processamento; DELETE solicita remoção. Cabeçalhos levam metadados, como Content-Type; corpo pode levar HTML/JSON. Status: 2xx sucesso, 3xx redirecionamento, 4xx erro do cliente, 5xx erro do servidor (ex.: 200, 404, 500).

HTTPS = HTTP protegido por TLS. Protege o tráfego; não garante que o site seja honesto. HTTP/2 permite multiplexar requisições em uma conexão. Um HTML pode gerar novas requisições para CSS, scripts e imagens.

```
https://exemplo.com:8443/aula?tema=css#box
```

https: esquema; **exemplo.com:** host; **8443:** porta; **/aula:** caminho; **tema=css:** consulta após ?; **box:** fragmento após # (não é enviado na requisição HTTP). Portas padrão: HTTP 80, HTTPS 443.

Caminhos: foto.jpg (mesma pasta), imagens/foto.jpg (subpasta), ../foto.jpg (pasta superior), /img/foto.jpg (raiz do site), https://... (URL completa). GitHub Pages hospeda conteúdo estático; não executa PHP como back-end. Abrir por file:// não equivale a servir por HTTP.

3. WordPress e ambiente local | S4-5

CMS: sistema de gestão de conteúdo. WordPress permite criar/gerenciar um site. **Tema:** apresentação; **plugin:** funcionalidade; **WooCommerce:** comércio eletrônico. **Posts:** publicações normalmente cronológicas, com categorias/tags; **páginas:** conteúdo como Sobre e Contato. Gutenberg é editor de blocos. MAMP fornece ambiente local com servidor web, banco e PHP; instalar localmente não publica o site na Internet.

Complemento de CSS: posicionamento

position: static (fluxo normal), relative (deslocamento preserva lugar original), absolute (fora do fluxo, referência usual no ancestral posicionado), fixed (referência usual na viewport), sticky (adere conforme rolagem e limites). float permite conteúdo fluir ao redor de uma caixa.

4. HTML: estrutura, semântica e mídia | S5-6

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <title>Minha página</title>
  <link rel="stylesheet" href="estilo.css">
  <script src="app.js" defer></script>
</head>
<body><h1>Título</h1><p>Texto</p></body>
</html>
```

head: metadados/referências; **title:** título da aba; **body:** conteúdo. h1-h6 organizam títulos por nível. Elemento = tags + conteúdo, quando aplicável; atributo fornece informação adicional. img, input e br são elementos vazios. Comentário: <!-- texto -->. Entidades: < representa <; & representa &.

Semântica: header (cabeçalho), nav (navegação), main (conteúdo principal), section (seção temática), article (conteúdo independente), aside (conteúdo relacionado), footer (rodapé). div e span são agrupadores genéricos. strong indica importância; em indica ênfase. id deve ser único; class pode se repetir e um elemento pode ter várias classes.

```
<a href="contato.html">Contato</a>
<img src="img/gato.jpg" alt="Gato dormindo">
<ol><li>Primeiro passo</li></ol>
```

href: destino do link; **src:** endereço do arquivo, não implica banco de dados; **alt:** alternativa textual contextual. Imagem decorativa: alt="". ol = lista ordenada; ul = não ordenada; li = item. dl/dt/dd = lista de descrições, termo e descrição. audio/video incorporam mídia; controls exibe controles; iframe incorpora outra página.

5. Tabelas e formulários | S7-8

```
<table>
 <tr><th>Produto</th><th>Preço</th></tr>
 <tr><td>Livro</td><td>30</td></tr>
</table>
```

tr: linha; **th:** célula de cabeçalho; **td:** célula de dados. colspan une colunas; rowspan une linhas. caption identifica a tabela; thead/tbody/tfoot agrupam suas partes. Tabelas representam dados tabulares, não devem ser usadas para montar o layout da página.

```
<form action="/cadastro" method="post">
 <label for="email">E-mail</label>
 <input id="email" name="email"
        type="email" required>
 <button type="submit">Enviar</button>
</form>
```

for do label = id do campo. name identifica o dado no envio; value é seu valor. placeholder é dica e não substitui label. action indica destino; method indica método. GET normalmente coloca dados na consulta da URL; POST normalmente no corpo: POST sozinho não criptografa dados.

Controles: text, email, password, number, checkbox (opções independentes), radio (uma opção por grupo de mesmo name), textarea, select/option. required exige preenchimento; min/max limitam valores. button type="button" não envia; type="submit" envia. Validação no navegador não substitui validação no servidor.

Leitura rápida antes de marcar a alternativa

Diferencie **nome de domínio de hospedagem**, **title de h1**, **id de class/name**, **src de alt** e **tema de plugin**. Leia se a questão pede definição, efeito visual ou comportamento. Números de pesquisas e rankings dos slides dependem do ano: não são verdades permanentes.

PROGRAMAÇÃO WEB

Revisão para múltipla escolha • Azul: conceitos-chave | Preto: explicações e exemplos

6. CSS: sintaxe, seletores e cascata | S7-8

```
p.aviso { color: blue; font-size: 1rem; }
```

Seletor escolhe elementos; declaração = propriedade: valor. CSS pode ser **externo** (link), **interno** (style) ou **inline** (atributo style). Não existe regra universal de que externo sempre perde para interno: a cascata decide.

Seletores: p (tag), .aviso (classe), #topo (ID), * (todos), [type="email"] (atributo), a:hover (pseudoclasse). **div p:** descendentes; **div > p:** filhos diretos; **h1, h2:** agrupamento; **p.aviso:** parágrafo com essa classe.

Cascata: considera origem/importância, camadas e especificidade, entre outros critérios. Entre regras normais do autor na mesma camada: **ID > classe/atributo/pseudoclasse > tag**. Persistindo empate, vence a última. Estilo inline normalmente supera seletores comuns; !important altera a prioridade e não é simples “mais especificidade”.

```
#x { color: blue; }
.aviso { color: red; }
p { color: green; }
/* <p id="x" class="aviso">: azul */
```

Herança: color e propriedades de fonte normalmente passam do pai aos filhos; margin, padding e border não. Uma declaração no próprio filho prevalece sobre valor apenas herdado. inherit solicita herança explicitamente.

7. Box model, display e layout | S8-9

Conteúdo → padding → border → margin. Padding é espaço interno; border é contorno com espessura; margin é espaço externo. Margem faz parte do modelo, mas não da largura até a borda. padding: 10px 20px significa 10 vertical e 20 horizontal; com quatro valores, ordem: cima, direita, baixo, esquerda.

content-box: width mede só conteúdo. Largura até as bordas = width + paddings laterais + bordas laterais. **border-box:** width já inclui padding e bordas. A margem fica fora nos dois casos.

```
width: 100px; padding: 10px;
border: 2px solid; margin: 5px;
/* content-box: 124px; com margens: 134px
   border-box: 100px; com margens: 110px */
```

block: normalmente quebra linha e aceita width/height. **inline:** acompanha o texto; width/height não se aplicam como em block. **inline-block:** acompanha a linha e aceita dimensões. **display:none:** não ocupa espaço; visibility:hidden oculta e preserva espaço.

8. Flexbox e responsividade | S8-9

display:flex no pai transforma filhos diretos em itens flexíveis. **flex-direction** escolhe eixo principal: row = horizontal; column = vertical, na escrita horizontal. **justify-content:** eixo principal. **align-items:** transversal. center centraliza; space-between distribui espaço entre itens. **flex-wrap:wrap** permite várias linhas; gap define espaçamento.

```
.pai { display:flex; flex-direction:row;
 justify-content:center; align-items:center;
 flex-wrap:wrap; gap:12px; height:200px; }
```

Unidades: px (CSS pixel), % (referência depende da propriedade), em (relativa à fonte; em font-size, à do pai), rem (fonte raiz), vw/vh (viewport). **Responsivo:** adapta layout ao espaço; use tamanhos flexíveis, max-width e media queries. Grid organiza linhas e colunas.

```
@media (max-width:600px) {
 .pai { flex-direction:column; }
}
/* head: meta name="viewport"
   content="width=device-width, initial-scale=1" */
```

9. JavaScript: diferenças para C++ | S9-10

Tipagem dinâmica: let x=1; x="oi" é válido. Tipos incluem number, string, boolean, undefined, null e object. **let:** reatribuição permitida; **const:** exige inicialização e impede reatribuição. Ambos têm escopo de bloco; var tem escopo de função, não de bloco.

```
const a = [10,20];
a[0] = 99; a.push(30); // permitido
// a = [1];            // erro
/* const fixa a referência, não o conteúdo */
```

Arrays: índices iniciam em 0, podem crescer e misturar tipos; length é tamanho; push adiciona ao fim; pop remove do fim; índice inexistente normalmente retorna undefined. Objeto agrupa propriedades: const p={nome:"Ana"}; p.nome acessa a propriedade.

= atribui; **==** pode converter tipos; **===** compara sem conversão. != e !== são as respectivas desigualdades. + soma ou concatena; % é resto; ** é potência. &&, || e ! são operadores lógicos. Em condicionais, 0, "", null, undefined, NaN e false são falsy; "0", [] e {} são truthy.

```
"5" + 2          // "52" (string)
Number("5") + 2  // 7 (number)
"5" - 2          // 3 (conversão numérica)
5 == "5"         // true
5 === "5"        // false
0 == false       // true
0 === false      // false
"texto" * 2      // NaN (not a number)
```

Funções: parâmetros recebem argumentos; return devolve resultado. Sem retorno explícito, o resultado é undefined. console.log exibe, não substitui return. if/else e laços seguem ideias familiares ao C++; atenção à conversão de tipos. Strings têm length; concatenação não é soma numérica.

```
function dobro(n) { return n * 2; }
const r = dobro(4); // r = 8
function mostrar(n) { console.log(n); }
const s = mostrar(4); // imprime 4; s=undefined
```

10. DOM, eventos e depuração | S3, S9-10

DOM: representação do documento como árvore de nós/objetos. querySelector retorna o primeiro correspondente ao seletor CSS ou null; querySelectorAll retorna uma coleção. Alterar DOM pode mudar a página sem comunicação com servidor.

```
const p = document.querySelector("#msg");
p.textContent = "Oi";     // substitui texto
p.textContent += " Ana"; // acrescenta texto
// const permite alterar propriedade do objeto
const b = document.querySelector(".botao");
b.addEventListener("click", tratar);
function tratar() { p.textContent = "Clicou"; }
```

tratar passa a função; **tratar()** executa imediatamente e passa seu retorno. textContent trata texto; innerHTML interpreta marcação HTML. addEventListener registra um tratamento de evento, como click, input ou submit.

Carregamento: script comum pode bloquear a análise do HTML e executar antes de elementos posteriores existirem. Script externo com **defer** baixa em paralelo, executa após análise do HTML, preserva ordem e antecede DOMContentLoaded. **async** executa quando disponível, sem garantir ordem nem DOM pronto. Script no fim do body encontra elementos anteriores.

Depuração: erro de sintaxe impede interpretar o código; erro de execução ocorre ao executar uma operação inválida; erro lógico produz comportamento incorreto. Console mostra mensagens e localização; breakpoints permitem pausar e inspecionar valores. Seleção retornando null pode indicar seletor errado ou elemento ainda inexistente.